Refugio Animal – versión 3

Análisis de la arquitectura y del código

La aplicación combina diccionarios para el inventario, clases POO para especies nativas, EspeciePersonalizada para especies dinámicas, un adaptador dict↔objeto, Tkinter para la UI y dos JSON de persistencia: especies_personalizadas.json e inventario.json.

1. Distinción arquitectónica

- Inventario de animales: diccionarios + inventario.json
- Especies nativas: clases Python (Perro, Gato, …)
- Especies personalizadas: EspeciePersonalizada + JSON
- Adaptador: conecta registros con objetos POO
- Tkinter: muestra fichas polimórficas generadas por los objetos

2. Estructura de carpetas

tarea1/ agrupa app.py, main.py, datos/, validaciones/, interfaz/, exportacion/, adjuntos/, muestras/, tests/ y docs/. El cuaderno arquitectura_refugio_animal_v3.ipynb documenta esta misma visión.

3. Punto de entrada

app.py añade la carpeta al sys.path, carga el inventario con cargar_inventario(), crea tk.Tk(), instancia GestorAnimales y arranca mainloop(). main.py solo delega en app.main().

4. Paquete datos/

Archivo	Responsabilidad
inventario.py	CRUD sobre Dict[int, dict]
persistencia_inventario.py	Carga/guarda inventario.json
modelos.py	ABC AnimalRefugio y especies
adaptador_poo.py	registro_a_animal / ficha
gestor_especies.py	Catálogo + JSON especies
ficha.py / adjuntos.py	Clínica y archivos

crear_animal valida con validar_datos (que consulta el catálogo) y asigna un ID. La UI persiste tras cada alta, edición, borrado o cambio de ficha clínica. El guardado es atómico (tmp + replace).

5. POO y ficha polimórfica

AnimalRefugio define propiedades encapsuladas y tres métodos abstractos. Las clases nativas y EspeciePersonalizada los implementan. generar_ficha() ensambla generales, comportamiento, alimentación y cuidados. Al seleccionar una fila, gestor.py convierte el dict con el adaptador y pinta la ficha sin if por especie.

6. Interfaz Tkinter

GestorAnimales coordina formulario, tabla, filtros, ficha POO, diálogo de especies, ficha clínica y exportación PDF. dialogo_especies.py no escribe JSON a mano: usa CatalogoEspecies. Los tests de GUI pasan

persistir=False para no ensuciar el JSON real.

7. Flujos

- Arranque: app → cargar_inventario → GestorAnimales → mainloop
- Alta: formulario → crear_animal → validar → guardar_inventario
- Selección: Treeview → adaptador → generar_ficha → paneles
- Especie: DialogoGestionEspecies → CatalogoEspecies → JSON → combobox

8. Validaciones y exportacion

validar_datos pertenece al dominio: nombre, especie activa (CatalogoEspecies), color, edad 0-30, peso finito en rango y estructuras clinicas. especie_previa permite conservar animales de especies desactivadas. La exportacion PDF (PdfFichas / fpdf2) lee diccionarios del inventario; no necesita objetos POO.

9. Dependencias y pruebas

Python >= 3.14 y fpdf2. Ejecucion: python app.py. Pruebas: python ejecutar_tests.py o unittest discover -s tests. test_persistencia_inventario.py cubre carga, guardado atomico y JSON corrupto. Pillow es opcional para imagenes / portapapeles.

10. Observaciones

- Especies e inventario son JSON distintos; los adjuntos van por carpeta ID.
- Hámster nativo lleva tilde; casefold no unifica con Hamster.
- Las pruebas GUI usan persistir=False para no ensuciar inventario.json.
- El cuaderno didactico no incluye clinica/PDF; la app de tarea1/ si.

11. Evolucion y defensa

Version	Aporte
1	CRUD + diccionarios + Tkinter
2	POO, adaptador, ficha polimorfica
3	Especies dinamicas + JSON
3+	Persistencia del inventario

Recorrido recomendado: crear Huron, registrar a Nico, seleccionar y explicar EspeciePersonalizada + generar_ficha; reiniciar la app para demostrar que inventario.json conserva los registros. Detalle ampliado: arquitectura_refugio_animal_v3.ipynb.